

Explication du firmware — TFC Système Multicanal

Guide pas-à-pas pour quelqu'un qui ne connaît pas le C++

Document préparé pour Samuel Mumbere Mathe — ULPGL

Juillet 2026

Table des matières

Introduction	2
Partie 1 — Les bases du C++ qu'il faut connaître	2
Une variable	2
Une constante	2
Un tableau	2
Une fonction	2
Une structure (struct)	3
Boucles et conditions	3
#include	3
Le pointeur * et la référence & (juste pour ne pas être surpris)	3
Partie 2 — config.h : les réglages du système	3
Partie 3 — Vue d'ensemble de main.cpp	4
C'est quoi une "tâche FreeRTOS" ?	4
C'est quoi un "mutex" ?	5
Partie 4 — Explication détaillée, bloc par bloc	5
Les includes (lignes 8-16)	5
La structure CanalEtat (lignes 18-29)	5
Les variables globales (lignes 31-37)	6
Sauvegarde et chargement NVS (lignes 42-66)	6
Tâche 1 — Acquisition des capteurs (lignes 71-83)	7
Tâche 2 — Calcul de la puissance (lignes 88-123)	7
Tâche 3 — Gestion des crédits (lignes 128-150)	8
Tâche 4 — Persistance LittleFS + NVS (lignes 155-181)	9
construireJsonEtat() (lignes 187-209)	9
Tâche 5 — Serveur API (lignes 216-271)	9
setup() — ce qui se passe au démarrage (lignes 273-309)	10
loop() — la boucle Arduino classique (lignes 311-313)	11
Partie 5 — Comment tout s'articule (résumé visuel)	11
Glossaire rapide	12

Introduction

Ce document explique, ligne par ligne et sans rien supposer connu, le fichier `src/main.cpp` du projet `firmware` (et son fichier de réglages `src/config.h`). Ce code fait tourner ton système à 5 canaux sur l'ESP32 : il lit les potentiomètres, calcule courant/puissance/énergie, gère les crédits, coupe les canaux, sauvegarde les données, et sert une API web.

Tu n'as pas besoin de tout retenir. L'objectif est qu'en relisant ton code tu puisses répondre si un membre du jury demande *“qu'est-ce que fait cette ligne ?”* — pas de devenir développeur C++ en une lecture.

Partie 1 — Les bases du C++ qu'il faut connaître

Le C++ est le langage utilisé pour programmer les microcontrôleurs comme l'ESP32 (via l'environnement Arduino/PlatformIO). Voici uniquement les notions utilisées dans ton fichier — pas tout le langage.

Une variable

Une variable, c'est une boîte étiquetée qui contient une valeur. En C++, il faut dire **de quel type** est la valeur avant de pouvoir l'utiliser :

```
int    age = 25;           // int = nombre entier
float  tension = 3.3;      // float = nombre a virgule (decimal)
bool   allume = true;      // bool = vrai ou faux (true/false)
String nom = "Samuel";     // String = texte
```

Dans ton fichier, tu verras énormément `int`, `float`, `bool`, `String` — ce sont juste les types de boîtes.

Une constante

`const` veut dire “cette boîte ne changera jamais après avoir été remplie”. `static const int PIN_TENSION = 25;` veut dire : *la broche du capteur de tension, c'est GPIO25, et ça ne changera pas pendant l'exécution.*

Un tableau

Un tableau, c'est une rangée de boîtes numérotées à partir de 0 :

```
int notes[3] = {12, 15, 9};
// notes[0] = 12
// notes[1] = 15
// notes[2] = 9
```

Dans `config.h`, `PIN_COURANT[NUM_CANAU]` est un tableau de 5 cases, une par canal, qui contient le numéro de broche GPIO de chaque potentiomètre.

Une fonction

Une fonction, c'est une recette : elle prend des ingrédients (les “paramètres”), fait des opérations, et parfois rend un résultat.

```
float doubler(float x) { // "doubler" prend un float, rend un float
    return x * 2;
}
```

void veut dire “cette fonction ne rend rien, elle fait juste une action”. La plupart des fonctions de ton fichier sont void.

Une structure (struct)

Une structure, c’est une fiche avec plusieurs champs regroupés sous un même nom. Ton fichier définit CanalEtat : la fiche d’un canal, avec son courant, sa puissance, son crédit, etc. — tout regroupé, comme une fiche de renseignements pour chaque locataire.

```
struct CanalEtat {
    float courant_A;
    float credit_USD;
};
CanalEtat canaux[5]; // 5 fiches, une par canal
canaux[0].credit_USD = 10; // remplir le champ credit_USD de la fiche 0
```

Boucles et conditions

- for (int i = 0; i < 5; i++) { ... } = *répète ce qui est entre accolades, pour i allant de 0 à 4 (5 fois)*. C’est ce qui permet de traiter les 5 canaux avec un seul bout de code au lieu d’écrire 5 fois la même chose.
- if (condition) { ... } = *si la condition est vraie, fais ceci*.
- for (;;) { ... } = *boucle infinie, ne s’arrête jamais* (utilisé dans chaque tâche : elle tourne pour toujours tant que l’ESP32 est allumé).

#include

#include <WiFi.h> = “j’ai besoin des outils du Wi-Fi, va les chercher”. C’est comme importer une bibliothèque de fonctions déjà écrites par quelqu’un d’autre (Espressif, la communauté Arduino...), pour ne pas réinventer la gestion du Wi-Fi, du JSON, etc. depuis zéro.

Le pointeur * et la référence & (juste pour ne pas être surpris)

Tu verras void *param ou AsyncWebServerRequest *request. Un pointeur, c’est une variable qui contient l’**adresse** d’une autre donnée, plutôt que la donnée elle-même — un peu comme une adresse postale au lieu de la maison. Tu n’as pas besoin de savoir manipuler ça toi-même pour ce TFC : retiens juste que * est présent quand une fonction accède à quelque chose d’externe (par exemple, la requête HTTP envoyée par un navigateur).

Partie 2 — config.h : les réglages du système

Ce fichier ne fait rien tout seul — c’est une liste de constantes utilisées partout ailleurs, regroupées à un seul endroit pour être faciles à changer.

```
#define NUM_CANAUX 5
```

#define remplace littéralement NUM_CANAUX par 5 partout dans le code avant la compilation. C’est le nombre de canaux de la maquette (1 bailleur + 4 locataires, cf. mémoire §3.6.1).

```
static const int PIN_COURANT[NUM_CANaux] = {36, 39, 34, 35, 32};
```

Le tableau des broches GPIO où sont branchés les 5 potentiomètres “courant” (qui simulent les ACS712). PIN_COURANT[0] = 36 = le canal 0 (bailleur) est lu sur la broche GPIO36.

```
static const int PIN_TENSION = 25;
```

La broche du potentiomètre “tension” (qui simule le ZMPT101B) — un seul, partagé par tous les canaux.

```
static const int PIN_RELais[NUM_CANaux] = {4, 5, 13, 14, 16};
```

Les broches qui commandent les LED (qui simulent les relais).

```
static const float COURANT_MAX_A = 20.0f;  
static const float TENSION_MAX_V = 250.0f;
```

Les valeurs maximales simulées : quand un potentiomètre est tourné à fond, on considère que ça représente 20 A (courant) ou 250 V (tension). Le f à la fin veut juste dire “c’est un float, pas un entier”.

```
static const float TARIF_USD_PAR_KWH = 0.20f;  
static const float CREDIT_INITIAL_USD = 0.05f;
```

Le prix du kWh (0,20 USD) et le crédit de départ de chaque canal (volontairement très bas — 0,05 USD — pour que la coupure automatique se déclenche vite pendant une démonstration, sans avoir à attendre longtemps).

```
static const uint32_t PERIODE_ACQUISITION_MS = 500;
```

uint32_t = encore un type de nombre entier (toujours positif, sur 32 bits — détail technique sans importance ici). Cette ligne dit : *on relit les capteurs toutes les 500 millisecondes (0,5 seconde)*.

Partie 3 — Vue d’ensemble de main.cpp

Avant les détails, voici le plan général du fichier :

1. **Includes** — les bibliothèques utilisées (Wi-Fi, serveur web, JSON, système de fichiers, mémoire NVS)
2. **La structure CanalEtat** — la “fiche” d’un canal
3. **Les variables globales** — la mémoire partagée du programme
4. **Deux fonctions de sauvegarde/chargement** (NVS)
5. **Cinq tâches FreeRTOS** — les 5 “ouvriers” qui tournent en parallèle (Acquisition, Calcul, Crédits, Persistance, Serveur API) — c’est le cœur du programme, détaillé en Partie 4
6. **setup()** — tout ce qui se lance une seule fois, au démarrage
7. **loop()** — presque vide ici, parce que tout le travail se fait dans les tâches, pas dans la boucle principale classique Arduino

C’est quoi une “tâche FreeRTOS” ?

Sur un Arduino classique, il n’y a qu’une seule boucle loop() qui répète tout, l’une après l’autre. L’ESP32 est plus puissant : il a **deux cœurs** de calcul, et un système appelé **FreeRTOS** qui permet de faire tourner plusieurs programmes “en même temps” (en réalité, ils se partagent le temps très rapidement, comme un chef qui bascule entre 5 casseroles sur le feu). Chaque tâche est une fonction avec une boucle infinie for(;;) à l’intérieur, qui tourne indépendamment des autres.

Ton système a 5 tâches — c’est comme avoir 5 employés qui travaillent en parallèle plutôt qu’un seul qui fait tout à la suite : un lit les capteurs, un autre calcule, un autre gère les crédits, un autre sauvegarde, un autre répond aux requêtes du téléphone.

C’est quoi un “mutex” ?

Les 5 tâches doivent parfois lire ou modifier les **mêmes** données (le tableau canaux). Si deux tâches essayaient d’écrire dedans exactement au même instant, ça pourrait corrompre les données (un peu comme deux personnes qui écrivent sur la même feuille en même temps).

Un **mutex** (mutexCanaux dans ton code) est comme une **clé unique** d’une salle : une tâche doit “prendre la clé” (xSemaphoreTake) avant d’entrer modifier les données, et la “rendre” (xSemaphoreGive) en sortant. Si une autre tâche veut entrer pendant ce temps, elle attend son tour. Ça garantit qu’il n’y a jamais deux tâches en train d’écrire au même moment.

Partie 4 — Explication détaillée, bloc par bloc

Les includes (lignes 8-16)

```
#include <Arduino.h>           // le coeur du framework Arduino (pinMode, digitalRead...)
#include <WiFi.h>               // gerer le point d'accès Wi-Fi de l'ESP32
#include <AsyncTCP.h>           // moteur reseau bas niveau utilise par le serveur web
#include <ESPAsyncWebServer.h> // le serveur web lui-meme (routes /admin, /api/etat...)
#include <ArduinoJson.h>        // creer et lire des donnees au format JSON
#include <LittleFS.h>           // systeme de fichiers dans la memoire flash de l'ESP32
#include <Preferences.h>        // memoire NVS (survit aux coupures/redemarrages)
#include "config.h"             // ton fichier de reglages (Partie 2)
```

Chaque #include avec des chevrons <...> vient d’une bibliothèque installée (par PlatformIO). #include "config.h" avec des guillemets veut dire “ce fichier est dans mon propre projet, pas une bibliothèque externe”.

La structure CanalEtat (lignes 18-29)

```
struct CanalEtat {
    int    brutCourant;
    float  tensionADC_V;
    float  courant_A;
    float  puissance_W;
    float  energie_kWh;
    float  energie_prev_kWh;
    float  credit_USD;
    bool   relaisFerme;
    String dernierEvenement;
    unsigned long dernierHorodatageMs;
};
```

C’est la fiche complète d’un **seul canal**. Chaque champ correspond directement à une colonne du Tableau 3.1/3.2/3.5/3.6 de ton mémoire :

Champ du code	Ce que ça représente
brutCourant	La valeur brute lue sur l'ADC (0 à 4095), avant tout calcul
tensionADC_V	Cette valeur brute convertie en volts (0 à 3,3V)
courant_A	Le courant calculé en ampères
puissance_W	La puissance calculée en watts
energie_kWh	L'énergie cumulée depuis la dernière recharge
energie_prev_kWh	La valeur d'énergie au tour précédent (sert à calculer la différence)
credit_USD	Le crédit restant du canal
relaisFerme	true = alimenté, false = coupé
dernierEvenement	Texte : "recharge", "coupure", "retablissement", "demarrage"
dernierHorodatageMs	À quel instant (en millisecondes depuis le démarrage) le dernier événement a eu lieu

Les variables globales (lignes 31-37)

```
static CanalEtat canaux[NUM_CANAUUX];
```

Un tableau de 5 fiches CanalEtat — une par canal. C'est LA donnée centrale de tout le programme : toutes les tâches lisent ou modifient ce tableau.

```
static int brutTension = 0;
static float tensionSecteur_V = 0.0f;
```

La lecture brute et la valeur convertie du capteur de tension (un seul, partagé — pas besoin d'un tableau).

```
static float tarifCourant = TARIF_USD_PAR_KWH;
```

Le tarif actuel, initialisé à la valeur de config.h mais qui pourrait être modifié plus tard (pas encore fait dans ce firmware).

```
static SemaphoreHandle_t mutexCanaux;
```

La "clé unique" expliquée en Partie 3.

```
static AsyncWebServer server(80);
```

Crée le serveur web, qui écoutera sur le port 80 (le port standard du web — c'est pour ça qu'on tape juste http://192.168.4.1 sans préciser de numéro de port dans le navigateur).

Sauvegarde et chargement NVS (lignes 42-66)

```
static void chargerEtatDepuisNVS() {
    Preferences prefs;
    for (int i = 0; i < NUM_CANAUUX; i++) {
        char ns[16];
        snprintf(ns, sizeof(ns), "canal%d", i);
        prefs.begin(ns, true);
        canaux[i].credit_USD = prefs.getFloat("credit", CREDIT_INITIAL_USD);
    }
}
```

```

    canaux[i].energie_kWh = prefs.getFloat("energie", 0.0f);
    prefs.end();
    ...
}
}

```

Au démarrage, cette fonction relit dans la mémoire NVS (une petite zone de la puce qui garde ses données même sans électricité) le crédit et l'énergie de chaque canal, pour reprendre exactement où on s'était arrêté avant la dernière coupure de courant — c'est directement lié à ton mémoire §1.5.3 : *“les coupures fréquentes justifient une sauvegarde non volatile”*.

- `char ns[16]` : une petite chaîne de caractères (16 cases) pour construire un nom du genre "canal0", "canal1"...
- `snprintf(ns, sizeof(ns), "canal%d", i)` : *écrit dans ns le texte “canal” suivi du numéro i*. C'est comme un `printf` (afficher du texte formaté) mais qui écrit dans une variable au lieu de l'écran.
- `prefs.getFloat("credit", CREDIT_INITIAL_USD)` : *va chercher la valeur stockée sous le nom “credit” ; si elle n'existe pas encore (premier démarrage), utilise CREDIT_INITIAL_USD par défaut*.

```
static void sauverCanalDansNVS(int i) { ... }
```

La fonction inverse : elle **écrit** le crédit et l'énergie actuels du canal `i` dans la mémoire NVS.

Tâche 1 — Acquisition des capteurs (lignes 71-83)

```

void TacheAcquisitionCapteurs(void *param) {
    for (;;) {
        if (xSemaphoreTake(mutexCanaux, pdMS_TO_TICKS(50)) == pdTRUE) {
            for (int i = 0; i < NUM_CANAL; i++) {
                canaux[i].brutCourant = analogRead(PIN_COURANT[i]);
            }
            xSemaphoreGive(mutexCanaux);
        }
        brutTension = analogRead(PIN_TENSION);
        vTaskDelay(pdMS_TO_TICKS(PERIODE_ACQUISITION_MS));
    }
}

```

Cette tâche ne fait qu'une seule chose, en boucle infinie (`for(;;)`) : - Elle prend la clé du mutex (attend au maximum 50 ms si quelqu'un d'autre l'a déjà) - Elle lit les 5 broches de courant avec `analogRead(...)` (fonction Arduino standard : lit une tension et la convertit en nombre de 0 à 4095) et range chaque résultat dans `canaux[i].brutCourant` - Elle rend la clé - Elle lit aussi la tension (en dehors du mutex, car un seul capteur, pas besoin de protection ici) - Elle attend 500 ms (`PERIODE_ACQUISITION_MS`) avant de recommencer

C'est la tâche la plus prioritaire (priorité 3 dans `setup()`) car acquérir les mesures régulièrement est le plus critique pour la précision.

Tâche 2 — Calcul de la puissance (lignes 88-123)

C'est ici que les valeurs brutes deviennent des grandeurs électriques utilisables. Reprenons le calcul principal :

```
float v = (canaux[i].brutCourant / (float)ADC_RESOLUTION) * ADC_VREF;
```

Convertit la valeur brute (0-4095) en volts (0-3,3V) : c’est une simple règle de trois. (float)ADC_RESOLUTION force la division à se faire en nombres décimaux (sinon C++ ferait une division entière et perdrait la précision).

```
float ecart = (v - (ADC_VREF / 2.0f)) / (ADC_VREF / 2.0f);
canaux[i].courant_A = fabsf(ecart) * COURANT_MAX_A;
```

Le signal simulé est centré sur la moitié de la tension d’alimentation (1,65V), correspondant à 0 A — comme le vrai ACS712 (cf. mémoire §1.3.1). ecart mesure de combien on s’écarte du centre, en proportion (-1 à +1). fabsf(...) = valeur absolue (on ne garde que l’écart, sans signe, car on ne s’intéresse qu’à la magnitude du courant ici). On multiplie ensuite par le courant maximal simulé (20A) pour avoir le résultat en ampères.

```
canaux[i].puissance_W = tensionSecteur_V * canaux[i].courant_A;
```

La formule $P = V \times I$ directement (cf. mémoire §1.3.3, avec $\cos(\varphi) \approx 1$ pour une charge résistive).

```
float dt_h = (PERIODE_CALCUL_MS / 1000.0f) / 3600.0f;
canaux[i].energie_kWh += (canaux[i].puissance_W / 1000.0f) * dt_h;
```

dt_h = la durée entre deux calculs, convertie en heures (500 ms → environ 0,000139 heure). += veut dire “*ajoute ceci à la valeur actuelle*” — on accumule l’énergie petit bout par petit bout, à chaque passage de la boucle, exactement comme le ferait un vrai compteur électrique.

La deuxième partie de la tâche (le Serial.println/Serial.printf) affiche un tableau dans le moniteur série toutes les 2 secondes (PERIODE_AFFICHAGE_MS) — **c’est ce tableau que tu lis pour remplir les Tableaux 3.5/3.6 de ton mémoire.**

Tâche 3 — Gestion des crédits (lignes 128-150)

```
float delta_kWh = canaux[i].energie_kWh - canaux[i].energie_prev_kWh;
if (delta_kWh > 0.0f) {
    canaux[i].credit_USD -= delta_kWh * tarifCourant;
    canaux[i].energie_prev_kWh = canaux[i].energie_kWh;
}
```

Calcule combien d’énergie a été consommée depuis le dernier passage de cette tâche (delta_kWh), et déduit le coût correspondant du crédit (-= = “*soustrait ceci de la valeur actuelle*”). On mémorise ensuite la nouvelle valeur d’énergie pour le prochain calcul de différence.

```
bool doitEtreFerme = canaux[i].credit_USD > 0.0f;
if (doitEtreFerme != canaux[i].relaisFerme) {
    canaux[i].relaisFerme = doitEtreFerme;
    canaux[i].dernierEvenement = doitEtreFerme ? "retablissement" : "coupure";
    canaux[i].dernierHorodatageMs = millis();
}
digitalWrite(PIN_RELAIS[i], canaux[i].relaisFerme ? HIGH : LOW);
```

Décide si le canal doit être alimenté (credit_USD > 0). != veut dire “*est différent de*” : si l’état a changé depuis le dernier passage (le canal vient d’être coupé ou vient d’être rétabli), on enregistre l’événement et l’heure. Enfin, digitalWrite(...) allume ou éteint physiquement la broche du relais/LED : HIGH = alimenté, LOW = coupé.

doitEtreFerme ? "retablissement" : "coupure" est un raccourci pour : *“si doitEtreFerme est vrai, utilise le texte ‘retablissement’, sinon utilise ‘coupure’”* — ça s’appelle un opérateur ternaire, c’est juste un if/else compact.

Tâche 4 — Persistance LittleFS + NVS (lignes 155-181)

```
for (;;) {
    vTaskDelay(pdMS_TO_TICKS(PERIODE_PERSISTENCE_MS));
    ...
    for (int i = 0; i < NUM_CANAL; i++) {
        sauverCanalDansNVS(i);

        JsonDocument doc;
        doc["canal"] = i;
        doc["energie_cumulee_kWh"] = canaux[i].energie_kWh;
        ...
        File f = LittleFS.open(path, "w");
        if (f) {
            serializeJson(doc, f);
            f.close();
        }
    }
}
```

Toutes les 5 secondes (PERIODE_PERSISTENCE_MS), cette tâche : 1. Sauvegarde le crédit et l’énergie de chaque canal en NVS (fonction déjà vue plus haut) 2. Construit un petit objet JSON (JsonDocument doc) avec l’historique du canal (format défini en Annexe A.3 de ton mémoire) 3. Ouvre un fichier /canal0.json, /canal1.json, etc. sur LittleFS en écriture ("w"), y écrit le JSON avec serializeJson, puis le ferme

JsonDocument, doc["canal"] = i : ArduinoJson te permet de construire un objet JSON comme un dictionnaire — doc["nom_du_champ"] = valeur. Quand tu appelles serializeJson(doc, ...), ça transforme cet objet en texte JSON classique, comme {"canal": 0, "energie_cumulee_kWh": 0.002, ...}.

construireJsonEtat() (lignes 187-209)

Cette fonction n’est pas une tâche, c’est une fonction “utilitaire” appelée par la tâche 5 à chaque fois qu’un téléphone demande l’état du système. Elle construit **un seul gros JSON** avec la tension secteur, le tarif, et un tableau (JsonArray) contenant la fiche complète de chaque canal. C’est exactement le JSON que lit l’application Flutter (api_client.dart, fonction fetchEtat()) et le JavaScript des pages web (app.js, fonction fetchEtat()) — les deux consomment la même source, ce qui garantit qu’ils affichent toujours les mêmes valeurs (cf. mémoire §2.10).

Tâche 5 — Serveur API (lignes 216-271)

```
server.on("/api/etat", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(200, "application/json", construireJsonEtat());
});
```

server.on(chemin, méthode, fonction) : *“quand quelqu’un visite cette adresse avec cette méthode HTTP, exécute cette fonction”*. Ici : si quelqu’un fait une requête GET sur /api/etat,

on répond avec le code 200 (= “tout va bien”, le code HTTP standard de succès) et le JSON de construireJsonEtat().

La partie `[](AsyncWebServerRequest *request) { ... }` est ce qu’on appelle une **fonction lambda** : une fonction définie directement sur place, sans lui donner de nom, parce qu’on ne l’utilise qu’à cet endroit. C’est comme écrire “*quand ça arrive, fais ceci*” directement, plutôt que de devoir écrire une fonction séparée ailleurs et la nommer.

```
server.on("/api/recharge", HTTP_GET, [](AsyncWebServerRequest *request) {
    if (!request->authenticate("bailleur", ADMIN_PASSWORD)) {
        return request->requestAuthentication();
    }
    ...
});
```

`request->authenticate(...)` vérifie le mot de passe **côté ESP32**, pas dans le navigateur — c’est le point de sécurité mentionné en mémoire §2.10 (“jamais en JavaScript”). Si le mot de passe est faux, `requestAuthentication()` renvoie une demande d’identification au navigateur, et le code s’arrête là (`return`) sans aller plus loin.

```
int idx = request->getParam("canal")->value().toInt();
float montant = request->getParam("montant")->value().toFloat();
```

Récupère les paramètres envoyés dans l’URL (par exemple `/api/recharge?canal=1&montant=5`) et les convertit en nombre (`toInt()`, `toFloat()`) — ils arrivent d’abord sous forme de texte.

La boucle suivante crée dynamiquement les routes `/L0`, `/L1`, ... `/L4` (une par canal), pour que chaque locataire ait sa propre adresse.

```
server.begin();
for (;;) {
    vTaskDelay(pdMS_TO_TICKS(60000));
}
```

`server.begin()` démarre vraiment le serveur (une seule fois). Ensuite, cette tâche n’a plus rien à faire activement — le serveur web répond tout seul en arrière-plan dès qu’une requête arrive (c’est ce que veut dire “événementiel” dans le commentaire du code) — donc la tâche se contente d’attendre 60 secondes en boucle, juste pour rester vivante sans consommer de ressources inutilement.

setup() — ce qui se passe au démarrage (lignes 273-309)

```
Serial.begin(115200);
```

Active la communication série (le moniteur série dans VS Code/Wokwi) à 115200 bauds (vitesse de transmission standard pour l’ESP32).

```
if (!LittleFS.begin(true)) { ... }
```

Monte le système de fichiers LittleFS. `true` veut dire “*si ça échoue, formate automatiquement*”. `!` devant veut dire “*si ce n’est PAS vrai*” — donc ce bloc s’exécute seulement si le montage a échoué.

```
for (int i = 0; i < NUM_CANAL; i++) {
    pinMode(PIN_RELAI[i], OUTPUT);
    digitalWrite(PIN_RELAI[i], LOW);
}
```

Configure chaque broche de relais/LED comme une **sortie** (OUTPUT, par opposition à une entrée qui lirait un signal), et l'éteint au démarrage par sécurité.

```
mutexCanaux = xSemaphoreCreateMutex();  
chargerEtatDepuisNVS();
```

Crée la "clé" du mutex, puis recharge le crédit/énergie sauvegardés.

```
WiFi.softAP(WIFI_SSID, WIFI_PASSWORD);
```

Démarré le point d'accès Wi-Fi de l'ESP32 (il devient lui-même un routeur Wi-Fi, cf. mémoire §2.9 — "sans dépendance à une connexion Internet externe").

```
xTaskCreatePinnedToCore(TacheAcquisitionCapteurs, "Acquisition", 4096, NULL, 3, NULL, 1);
```

C'est la ligne qui **crée** et **démarré** chaque tâche. Les paramètres, dans l'ordre : 1. Le nom de la fonction à exécuter en boucle 2. Un nom texte pour la reconnaître dans les outils de debug 3. La taille de la pile mémoire allouée à cette tâche (4096 octets) 4. Un paramètre optionnel passé à la tâche (NULL = aucun ici) 5. La priorité (plus le chiffre est grand, plus la tâche est prioritaire quand plusieurs veulent s'exécuter en même temps) 6. Un identifiant de tâche optionnel (NULL = non utilisé) 7. Le cœur du processeur sur lequel la coincer (0 ou 1 — l'ESP32 en a deux, cf. mémoire §1.4.2)

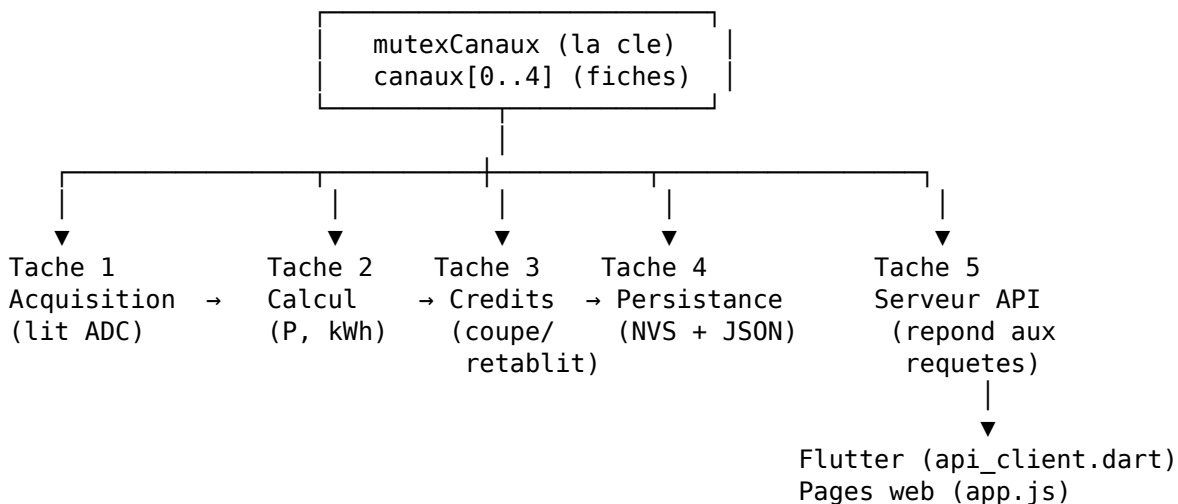
C'est la traduction directe de l'Annexe A.1 de ton mémoire.

loop() — la boucle Arduino classique (lignes 311-313)

```
void loop() {  
  vTaskDelay(pdMS_TO_TICKS(1000));  
}
```

Sur un programme Arduino simple, tout le travail se ferait ici. Mais comme tout ton travail est déjà fait par les 5 tâches créées dans setup(), loop() n'a plus rien à faire — elle attend juste 1 seconde en boucle, sans jamais s'arrêter (FreeRTOS a besoin qu'elle existe, même vide, car c'est elle-même une tâche cachée créée automatiquement par le framework Arduino).

Partie 5 — Comment tout s'articule (résumé visuel)



Chaque tâche lit et/ou modifie les mêmes fiches canaux[], toujours en passant par le mutex pour ne jamais se marcher dessus. C'est exactement l'architecture décrite au chapitre 2.6 de ton mémoire — ce document ne fait que “dérouler” en langage courant ce que le code fait ligne par ligne.

Glossaire rapide

Mot	Signification simple
ADC	Convertisseur qui transforme une tension (analogique) en nombre (0-4095)
GPIO	Une broche physique de l'ESP32
Tâche (task)	Un mini-programme qui tourne en boucle, en parallèle des autres
Mutex	Une clé unique qui empêche deux tâches d'écrire en même temps
JSON	Un format texte standard pour échanger des données structurées
NVS	Une petite mémoire de la puce qui garde ses données sans électricité
LittleFS	Un vrai système de fichiers (comme des dossiers/fichiers) sur la puce
Route (server.on)	Une adresse web à laquelle le serveur répond (ex : /admin)
Lambda ([](...){...})	Une fonction écrite directement sur place, sans nom
static	La variable garde sa valeur d'un appel de fonction à l'autre
const	La valeur ne change jamais après sa création